

AULA 7

Ações: tarefas longas, feedback e cancelamento

Etapa 4 · terça, 01/09/2026 · sala SJ205 · Prof. Dácio Moreira de Souza



O que vocês estão vendo?

(a webcam já está rodando — nenhum slide ainda)



**Ele não está seguindo nada.
Está dizendo para onde a câmera
TERIA que ir.**

A seta é um comando de motor que ainda não tem motor.

Demo 1 — a seta e a mensagem são a MESMA coisa

```
ros2 launch demo_visual demo1.launch.py
```

```
# noutro terminal, sem parar o primeiro:
```

```
ros2 topic echo /demo/comando
```

```
acao: 'esquerda'
```

```
erro_x: -0.42
```

```
motivo: 'alvo a esquerda do centro'
```

Quem desenha a seta é um consumidor da mensagem. Trocar esse consumidor por um driver de tração não muda uma linha de quem enxerga.

Demo 2 — a folha que chama um serviço

O que o aluno faz

- levanta a folha para a câmara
- triângulo → /robo/parar
- quadrado → /robo/seguir
- círculo → /robo/girar
- cruz → /robo/emergencia

O que está plantado ali

- a folha VERMELHA não dispara nada
- → cor e forma são pistas independentes
- folha levantada 10 s dispara UMA vez
- → debounce: 3 quadros + 3 s de descanso
- → o que separa detectou de DECIDIU

O que as duas demos realmente mostram

- Não são sobre visão computacional. São sobre o FORMATO do comando
- seta redesenhada a cada quadro -> fluxo contínuo, ninguém espera -> TÓPICO
- folha levantada uma vez -> evento pontual, com confirmação -> SERVIÇO
- "vá até a bancada e me avise" -> demora, informa, cancela -> ACTION
- Os dois primeiros vocês já sabem. O terceiro é hoje



Percepção produz observação a 30 Hz. Decisão não pode acontecer a 30 Hz.

Onde você põe essa barreira é decisão de projeto — e ela tem nome: debounce.

A Aula 6 revista, com o demo na tela

- ComandoMovimento é interface própria para separar PERCEBER de AGIR
- Quem enxerga não sabe o que vem depois; quem age não sabe como foi detectado
- Vermelho ocupa as DUAS pontas do matiz (0–10 e 170–180). Uma faixa só: "às vezes detecta"
- Saturação não significa nada sem luz: preto ruidoso mede $S=200$. Use V
- /vision/deteccoes não mudou quando a fonte virou webcam — interface bem escolhida



Hoje o seu robô ganha a capacidade de ser interrompido.

É a terceira forma de comunicação — e o alicerce de toda a navegação.

Primeiro, olhar para trás: o TP1 foi entregue

- O que você descobriu tarde demais? Quase nunca é técnico — é decisão adiada
- Qual gate você marcou [x] sem estar confortável? Esse volta no TP3
- O que faria diferente no TP2? Uma frase concreta, não um parágrafo genérico
- Escreva na seção de retrospectiva do PROJETO.md — agora, enquanto está fresco

PARTE 1

As três formas, e onde cada uma quebra



**"Vá até a bancada
e me avise quando chegar."**

Como você implementa isso com o que já sabe?

O que já temos, e o que falta

Tópico e serviço

- tópico: publica e torce — ninguém confirma nada
- serviço: trava o cliente 30 s, em silêncio
- desistiu no meio? o robô continua indo
- serviço não tem como ser cancelado

Action

- confirma que ACEITOU — ou rejeita
- publica progresso enquanto anda
- para se você mandar parar
- pode terminar cedo se já bastou

A regra prática

- Resposta demora mais que uma fração de segundo → serviço é a escolha errada
- Vai querer saber COMO ESTÁ INDO, ou MANDAR PARAR → action
- Todo o resto → tópico e serviço continuam certos
- Action custa código de verdade. Não use onde não precisa

Baixar o material desta aula

```
rm -rf /tmp/PBRoboticos_prof_dacio
cd /tmp && git clone --depth 1 \
    https://github.com/Prof-Dacio-INFNET/PBRoboticos_prof_dacio.git

# pb_interfaces vem DE NOVO: ele mudou, ganhou a action
cp -r ../aula06-interfaces/pb_interfaces \
    ../aula06-interfaces/aula06_percepcao \
    ../aula07-acoas/aula07_acoas \
    ~/projeto-pb-SEU-USUARIO/ros2_ws/src/
```

Interfaces primeiro, sempre: `colcon build --packages-select pb_interfaces`, depois `source`, depois o resto.

PARTE 2

Anatomia de um .action

Três blocos, separados por ---

```
# ----- Goal: o que se pede -----  
float32 duracao_s  
string classe_alvo  
uint32 minimo_para_parar    # 0 = vai ate o fim  
---  
# ----- Result: o que volta no FIM -----  
uint32 total_visto  
float32 confianca_media  
string resumo  
---  
# ----- Feedback: o que volta DURANTE -----  
float32 decorrido_s  
uint32 vistos_ate_agora  
float32 progresso
```

O pacote de interfaces CRESCE

- O .action mora no mesmo pb_interfaces da Aula 6 — não se cria pacote por aula
- Uma linha no rosidl_generate_interfaces: "action/VarrerCena.action"
- action_msgs vira dependência nos DOIS lugares: package.xml e DEPENDENCIES
- Toda action carrega identificador e status de objetivo — vem de action_msgs
- minimo_para_parar existe para a tarefa TERMINAR CEDO. Longa $\neq$ duração fixa

PARTE 3

O servidor: três decisões, não uma

Cada callback responde a uma pergunta diferente

Antes de trabalhar

- goal_callback: aceita ou REJEITA
- recusa pedido impossível, fora de faixa, perigoso
- rejeitado $\neq$ falhou: olhou e disse não, sem gastar nada

Durante

- cancel_callback: aceita ou recusa o cancelamento
- é DECISÃO, não automatismo
- braço no meio de um movimento pode precisar recolher antes

A ordem dentro do laço importa mais que o conteúdo

```
while True:
    if goal_handle.is_cancel_requested:      # 1. SEMPRE primeiro
        goal_handle.canceled()              #   sem isto o cliente pendura
        return resultado_parcial

    ...                                     # 2. o trabalho
    if condicao_de_sucesso:                  # 3. sucesso antecipado
        goal_handle.succeed()
        return resultado
    goal_handle.publish_feedback(fb)         # 4. feedback
    time.sleep(PERIODO_S)                   # 5. dormir um pouco
```

Checar cancelamento no FIM do laço faz o servidor demorar um ciclo inteiro para obedecer. Num robô móvel, isso é segurança.

PARTE 4

O bug que não dá erro



**Tudo certo no código da action.
Pede cancelamento.
Não cancela.
E não aparece erro nenhum.**

A causa está fora do código da action

- Por padrão o rclpy roda com um executor de UMA thread só
- O `execute_callback` ocupa essa thread inteira até terminar
- O pedido de cancelamento chega, entra na fila... e espera a thread ficar livre
- A thread só fica livre quando a tarefa acaba sozinha
- `is_cancel_requested` nunca vira verdadeiro, porque nada nunca o setou

A cura são duas linhas — e as duas são necessárias

```
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor

# 1. no ActionServer:
callback_group=ReentrantCallbackGroup()

# 2. no main():
rclpy.spin(no, executor=MultiThreadedExecutor())
```

O comportamento ausente não produz erro.

Testes procuram erro. Comportamento ausente só aparece se você o exercitar de propósito — por isso o G2.3 pede cancelamento DEMONSTRADO.

PARTE 5

Do lado do cliente, sem escrever cliente

A aula inteira em uma flag

```
ros2 action list
ros2 action info /varrer_cena -t

ros2 action send_goal /varrer_cena \
  pb_interfaces/action/VarrerCena \
  "{duracao_s: 15.0, classe_alvo: 'tomate_maduro',
  minimo_para_parar: 0}" --feedback
```

Sem `--feedback` você vê um comando que demora e devolve algo: indistinguível de um serviço lento. Com ele, o progresso chega DURANTE.



Matar o cliente e cancelar o objetivo são coisas diferentes.

É assim que se descobre, tarde, que o robô continuou andando depois que o terminal fechou.

Onde isso vira TP2 — e onde vira navegação

- G2.2: action com servidor respondendo · vence 12/09
- G2.3: feedback e cancelamento DEMONSTRADOS · vence 16/09
- A armadilha do documento de gates: "a Action como um publisher com nome bonito"
- NavigateToPose, do Nav2, É uma action: objetivo = pose, feedback = distância restante
- Na Etapa 5 você aprende navegação usando uma coisa que já entende

Antes da próxima terça

- Tarefa: uma action que faça sentido no SEU domínio, com parada antecipada
- Rejeite alguma coisa de propósito — servidor que aceita tudo não exercita metade
- GRAVE a evidência do cancelamento: o gate pede demonstração, não código
- Aula 8, 08/09: percepção veicular — detecção treinada e métrica declarada (G2.4)